# Stack & Queue — Pattern Recognition & Universal Notes

These notes are meant for quick revision before solving any stack or queue problem. They focus on recognizing when these data structures apply, common problem patterns, and reusable templates.

---

# 1. How to Recognize a Stack Problem

Typical signals:

- "Next greater / smaller element"
- "Balanced parentheses"
- "Undo / backtracking behavior"
- "Evaluate expression"
- "Monotonic behavior"

Core intuition:

A **stack stores elements in LIFO order (Last In First Out)**.

This is useful when we need to process the **most recent unfinished work first**.

---

# 2. Basic Stack Operations

```
stack<int> st;

st.push(x);
st.pop();
st.top();
st.empty();
```

Time complexity:

```
O(1)
```

---

# 3. Balanced Parentheses Pattern

Classic stack problem.

Idea:

Push opening brackets.

When closing bracket appears, match with top.

Template:

```
stack<char> st;

for (char c : s) {

    if (c == '(' || c == '{' || c == '[')
        st.push(c);

    else {
        if (st.empty()) return false;

        char t = st.top();
        st.pop();

        if (!match(t, c))
            return false;
    }
}

return st.empty();
```

Used in:

- Valid parentheses
- Expression validation

---

# 4. Monotonic Stack Pattern

Very important pattern.

Stack keeps elements in **increasing or decreasing order**.

Used when we need **next greater / smaller element**.

Example idea:

When current element is larger than stack top, we resolve previous elements.

Template:

```
stack<int> st;

for (int i = 0; i < n; i++) {

    while (!st.empty() && arr[st.top()] < arr[i]) {
        int idx = st.top();
        st.pop();

        // arr[i] is next greater for idx
    }

    st.push(i);
}
```

Used in:

- Next Greater Element
- Daily Temperatures
- Stock Span

---

# 5. Stack for DFS Simulation

Stack can simulate recursion.

Template:

```
stack<TreeNode*> st;
st.push(root);

while (!st.empty()) {

    TreeNode* node = st.top();
    st.pop();

    if (node->right) st.push(node->right);
```

```
    if (node->left) st.push(node->left);
}
```

Used for:

- iterative tree traversal

---

# 6. Queue Fundamentals

Queue follows **FIFO (First In First Out)**.

Useful when processing elements **in order of arrival**.

Operations:

```
queue<int> q;

q.push(x);
q.pop();
q.front();
q.empty();
```

---

# 7. BFS Pattern (Queue)

Most common queue use.

Used in:

- trees
- graphs
- shortest path in unweighted graph

Template:

```
queue<int> q;
q.push(start);

while (!q.empty()) {

    int node = q.front();
```

```
    q.pop();

    for (auto next : neighbors[node]) {
        q.push(next);
    }
}
```

---

# 8. Level Order Traversal Pattern

Queue processes nodes level by level.

Template:

```
queue<TreeNode*> q;
q.push(root);

while (!q.empty()) {

    int size = q.size();

    for (int i = 0; i < size; i++) {

        TreeNode* node = q.front();
        q.pop();

        if (node->left) q.push(node->left);
        if (node->right) q.push(node->right);
    }
}
```

Used in:

- level order traversal
- zigzag traversal

---

# 9. Sliding Window with Deque

Deque allows push/pop from both sides.

Used for problems like **sliding window maximum**.

Template:

```
deque<int> dq;

for (int i = 0; i < n; i++) {

    while (!dq.empty() && dq.front() <= i - k)
        dq.pop_front();

    while (!dq.empty() && arr[dq.back()] < arr[i])
        dq.pop_back();

    dq.push_back(i);

    if (i >= k - 1)
        answer.push_back(arr[dq.front()]);
}
```

---

# 10. Mental Checklist Before Coding

Ask yourself:

1. Do I need LIFO behavior? → Stack
2. Do I need FIFO behavior? → Queue
3. Is this a next greater/smaller problem? → Monotonic stack
4. Is this level-by-level traversal? → Queue
5. Is this sliding window max/min? → Deque

---

# 11. Pattern Recognition Summary

Most stack/queue problems fall into these categories:

| Pattern | Example |
|---|---|
| Parentheses matching | Valid parentheses |
| Monotonic stack | Next greater element |
| Expression evaluation | Reverse polish notation |
| BFS traversal | Graph BFS |
| Level order traversal | Binary tree level order |

| Pattern | Example |
|---|---|
| Sliding window | Sliding window maximum |

Recognizing the pattern quickly is the key.

---

# 12. Universal Templates

Stack push/pop

```
st.push(x);
st.pop();
```

Monotonic stack

```
while (!st.empty() && condition)
    st.pop();
```

Queue BFS

```
while (!q.empty())
```

Deque sliding window

```
pop_front / pop_back
```

These patterns solve the majority of stack and queue problems.